

Transformer II: posição, normalização e o bloco moderno

Como o mecanismo da Aula 6 vira um bloco que treina com 48 camadas.

Seu nome

seuemail@cesar.org.br

Setembro / 2026

O SINTOMA QUE A AULA 6 DEIXOU NA TELA

Duas frases opostas, as mesmas representações

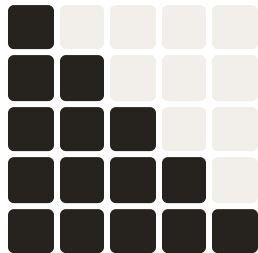


Não é erro de treino nem caso raro: é o que a fórmula faz, sempre. E nenhuma tarefa que dependa de ordem — tradução, geração, qualquer coisa com sintaxe — funciona sobre isso.

O mecanismo que montamos na aula passada não sabe que ordem existe. E isso não se corrige com if.

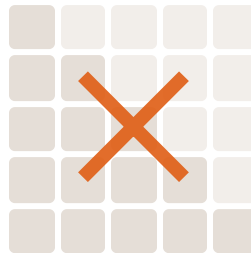
POR QUE A INTUIÇÃO SOZINHA ERRA AQUI

“Mas a máscara causal não resolve isso?”



quem pode ler quem

A máscara apaga metade da matriz. É permissão de leitura.



quanto dista de quem

Essa informação não está lá. Dois tokens visíveis são indistinguíveis quanto à distância.

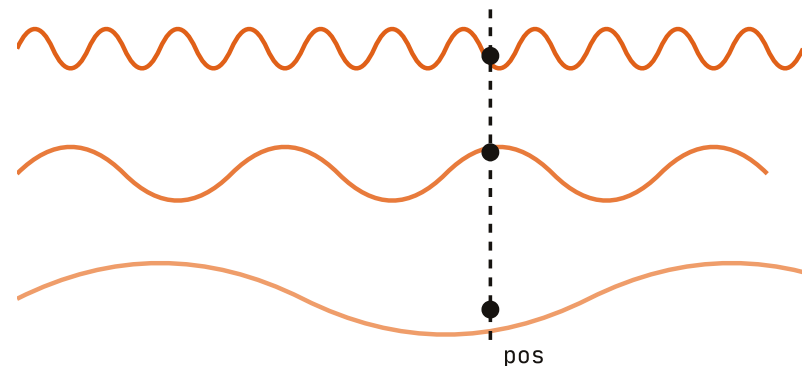
E há o caso que encerra a discussão: no **encoder não existe máscara nenhuma**, e lá o problema aparece puro.

| **Máscara é sobre permissão de leitura. Posição é sobre distância. Só uma delas está no mecanismo.**

Um relógio de muitos ponteiros

```
PE(pos, 2i)   = sin(pos / 10000^(2i/d)) # pares  
PE(pos, 2i+1) = cos(pos / 10000^(2i/d)) # ímpares  
x_pos = x_token + PE(pos)                # soma na entrada
```

As primeiras dimensões oscilam rápido e distinguem vizinhos; as últimas oscilam devagar e distinguem regiões da frase. Juntas, dão um código único por posição.



rápido → vizinhos

lento → regiões

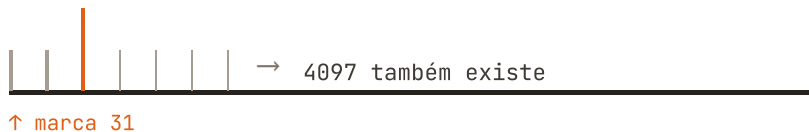
Fundamento: o produto interno $PE(pos) \cdot PE(pos+k)$ depende só de k . → Apêndice A.1 (slide 15)

Em vez de ensinar a rede a contar, o paper de 2017 deu um relógio de vários ponteiros para cada posição.

A SEGUNDA RESPOSTA, E O SINTOMA QUE ELA PRODUZ

O modelo que vira lixo no token 4097

Em vez de calcular o código, **aprender** uma tabela de vetores, um por posição. Mesma soma na entrada; PE passa a ser parâmetro, não função.



Senoidal é uma régua

Qualquer marca é calculável, dentro ou fora do treino.

```
IndexError: index 4096 is out of bounds
```

linha 4095

linha 4096

linha 4097 – vazia



Aprendida é uma gaveta de etiquetas

A 4097 existe se alguém imprimiu. E não há noção de distância embutida: o modelo aprende que 31 é vizinha de 30 par a par, pelos dados.

Um modelo treinado com 4096 posições produz lixo a partir da 4097. Não fica ruim: não existe entrada na tabela.

Girar Q e K em vez de somar posição

```
 $\theta_i = 10000^{(-2i/d)}$       # uma freq. por par
```

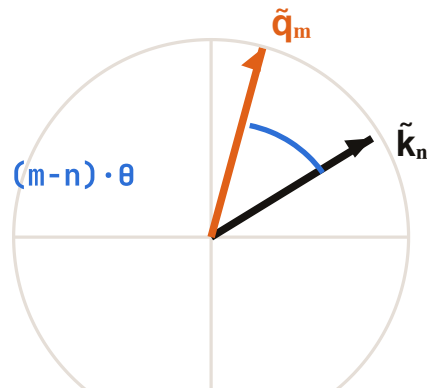
```
 $R(a) = \begin{bmatrix} \cos a & -\sin a \\ \sin a & \cos a \end{bmatrix}$   # rotação 2D
```

```
 $\langle R(m \cdot \theta)q, R(n \cdot \theta)k \rangle = f(q, k, m-n)$ 
```

Q → RoPE → score → softmax

K → RoPE ↗

V ————— passa reto — nunca rotacionado



Medição 4: distância 2 dá 0.676869 na posição 3 e na posição 5000. → Apêndice A.2 (slide 16)

Não interessa onde cada ponteiro está no mostrador. Interessa o ângulo entre eles — e o ângulo só depende da distância.

POR QUE ROPE VENCEU, E O QUE ELE NÃO RESOLVE

Quatro razões, e uma honestidade

01

Relativa, e no lugar certo

Entrega distância dentro do score, onde a comparação entre dois tokens acontece.

02

Zero parâmetro treinável

Rotação determinística, calculada na hora. Nenhuma tabela para aprender ou estourar.

03

Reinjetada em cada camada

Nas outras, a posição entra uma vez e precisa sobreviver a 30 blocos de mistura.

04

Alavanca de contexto

A janela pode ser estendida reescalando as frequências — tabela aprendida não permite.

HONESTIDADE

Reescalar RoPE **estica** a janela; não faz o modelo *usar* bem o que entrou nela. Janela grande e memória confiável são coisas diferentes. → [Aula 10](#)

Posição relativa, zero parâmetro, reinjetada em toda camada. É difícil competir com isso.

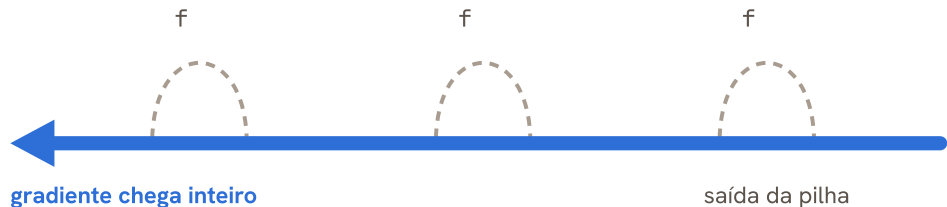
[DEMO] MEDIR A POSIÇÃO

Quatro medições, quinze linhas de NumPy

- 01 Sem posição — o script da Aula 6 com `--shuffle`
- 02 Senoidal — $PE(pos) \cdot PE(pos+3)$ para $pos = 0, 5, 100, 1000$
- 03 Aprendida — tabela de 4096 linhas, acesso à linha 4096
- 04 RoPE — score de $(3,1)$, $(10,8)$, $(500,498)$, $(5000,4998)$

As três estratégias medidas lado a lado — e uma delas produz o mesmo número dentro e fora do comprimento de treino.

Doze blocos treinam, quarenta e oito divergem



$$y = x + f(x) \quad \# \text{ o sub-bloco aprende um delta}$$

A derivada passa a ter **dois termos**: um atravessa o ramo f , o outro é a **identidade** — não multiplica nada, não encolhe nada.

$$0.8^{40} \approx 1.3 \cdot 10^{-4}$$

Com 40 blocos e fator típico 0.8 por bloco, o sinal de erro chega às primeiras camadas quatro ordens de grandeza menor. É o produto da Aula 5, agora ao longo da **profundidade**.

Com residual, um bloco que aprende $f = 0$ é **inofensivo** — e é isso que permite empilhar sem medo. → Apêndice A.3 (slide 17)

O gradiente ganha dois caminhos de volta, e um deles é a identidade. Esse não se dilui em quarenta camadas.

LayerNorm centra e escala. RMSNorm só escala.

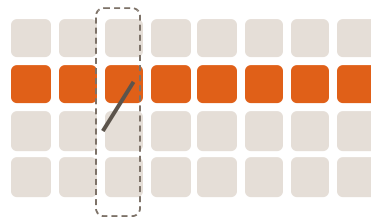
$$\text{LayerNorm}(x) = \gamma \odot (x - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta$$

$$\text{RMSNorm}(x) = \gamma \odot x / \sqrt{(\text{mean}(x^2) + \epsilon)}$$

O que a segunda economiza: uma passagem a menos sobre o vetor e **metade dos parâmetros da norma** (só γ). O que perde: a invariância a deslocamento. Na prática a qualidade é equivalente — é para onde os LLMs abertos convergiram.

Por que normalizar: a escala cresce a cada soma residual. → Apêndice A.4 (slide 18)

Isto não é BatchNorm



■ a **linha** : as dimensões de um exemplo

□ a **coluna** : não é isto

Por isso a norma é indiferente ao tamanho do lote e funciona igual ao gerar um token só.

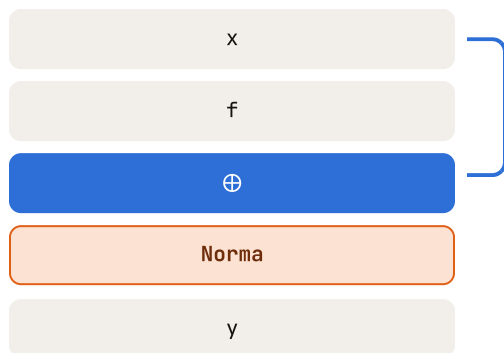
Se o que atrapalhava o treino era a magnitude do vetor, e não o centro dele, centrar era trabalho que ninguém tinha cobrado.

ONDE A NORMA ENTRA MUDA O TREINO

Mesmas peças, duas ordens, dois regimes de treino

$$y = \text{LN}(x + f(x))$$

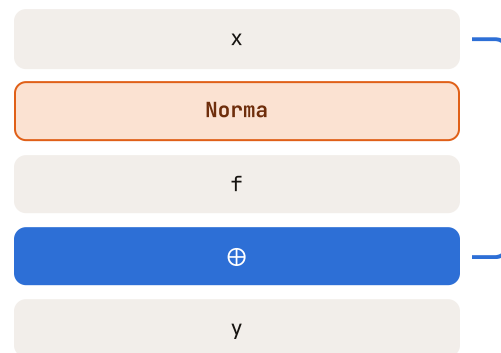
pós-norm — 2017



A soma passa pela norma: a autoestrada é **reescalada a cada camada**. Pilha profunda exige aquecimento cuidadoso do learning rate.

$$y = x + f(\text{LN}(x))$$

pré-norm — LLMs modernos



A norma entra dentro do ramo: o caminho residual fica **limpo** da entrada até a saída da pilha. Treina com muito menos truque.

O detalhe que falta em quase todo diagrama

Com pré-norm é **obrigatória uma norma final** depois do último bloco, antes da cabeça de saída — a última soma residual não passou por norma nenhuma. Esquecida, o treino diverge *sem mensagem de erro*.

O preço, dito honestamente: em modelos **rasos** o pós-norm costuma render um pouco mais. A escolha se inverteu quando a profundidade cresceu.

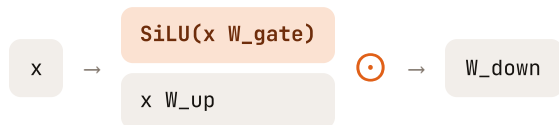
→ Apêndice A.5 (slide 19)

Mesmas peças, ordem diferente: uma pilha precisa de aquecimento na unha para não explodir, a outra simplesmente treina.

ONDE ESTÃO DOIS TERÇOS DOS PESOS

A parte que não olha para nenhum vizinho

```
FFN(x) = W_2 · GELU(W_1 x) # 2017, d_ff = 4·d
SwiGLU(x) = (SiLU(x W_gate) ⊙ (x W_up)) W_down
```



A atenção **mistura** posições; o feed-forward **transforma** cada posição isolada. Um ramo decide o quê, o outro decide quanto passa.

4·d² atenção8·d² feed-forward

atenção (d = 1024) 4,2 M

feed-forward 8,4 M

bloco ≈ 12·d² 12,6 M

Dobrar d **quadruplica** os parâmetros. E o 8/3 das configurações é o número que paga a terceira matriz. → A.6 (slide 20)

Dois terços dos pesos do bloco estão justamente na parte que não olha para nenhum vizinho.

O BLOCO MODERNO, CONSOLIDADO

2017 → hoje: cinco trocas, nenhuma no mecanismo

PEÇA	TRANSFORMER DE 2017	BLOCO MODERNO
Posição	senoidal somada na entrada	RoPE em Q e K, a cada camada
Onde a norma entra	pós-norm: $\text{LN}(x + f(x))$	pré-norm + norma final
Qual norma	LayerNorm (média e β)	RMSNorm (só escala)
Feed-forward	GELU, $d_{\text{ff}} = 4 \cdot d$	SwiGLU, $d_{\text{ff}} \approx 8/3 \cdot d$
Como a pilha é montada	encoder-decoder	decoder-only (Aula 8)

Sintomas que motivaram: lixo além do treino · divergência ao empilhar · custo · qualidade a paridade de parâmetros.

```
h = x + MHACausal(RMSNorm(x), rope)
y = h + SwiGLU(RMSNorm(h))
# ... N blocos ...
logits = Linear(RMSNorm(y_final))
```

Duas normas e duas somas por bloco. É isso.

NÃO MUDOU EM OITO ANOS

```
softmax(QKT/vd_k)V · multi-head
residual · FFN por posição
```

Cinco trocas em oito anos e nenhuma delas foi no mecanismo. Trocaram a suspensão, não o motor.

[EXERCÍCIO] ACHAR A PEÇA ERRADA

8 minutos, três sintomas, uma peça cada

01

Uma pilha de 48 blocos diverge nos primeiros 200 passos. A mesma receita, com 12 blocos, treina até o fim. Sem NaN antes da divergência; as primeiras camadas mal se movem.

peça

evidência que confirma

02

Um modelo responde bem em prompts curtos e produz texto incoerente assim que o prompt passa de um certo comprimento — sempre o mesmo comprimento, em qualquer assunto.

peça

evidência que confirma

03

Um time aplicou a rotação posicional em Q, K e V. O modelo treina, e a representação de uma mesma frase muda conforme ela aparece no começo ou no fim do prompt.

peça

evidência que confirma

PEÇAS PARA CONSULTA

RoPE

PE aprendida

residual

pré-norm

RMSNorm

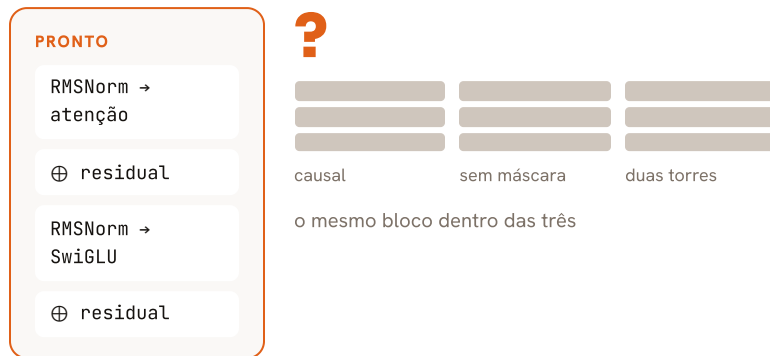
SwiGLU

08:00

Nenhuma das três se resolve derivando. As três se resolvem sabendo o que cada peça do bloco compra.

FECHAMENTO

O bloco está pronto. Falta decidir o que cada token pode enxergar.



A Aula 6 construiu o **mecanismo**; a Aula 7 construiu o **bloco**. O que faz um modelo classificar e outro escrever não está no bloco — está em como a pilha é conectada. Segundo fio: a atenção custa $O(n^2)$.

LEITURA DA SEMANA · PERGUNTA DIRIGIDA

Su et al., *RoFormer* (arXiv 2104.09864) — seções de formulação e de propriedades.

Em que ponto exato do cálculo do score a posição entra, e por que isso entrega posição relativa sem nenhum parâmetro novo?

posição	senoidal → RoPE em Q e K
norma	pós-norm → pré-norm + final
qual norma	LayerNorm → RMSNorm
feed-forward	GELU 4·d → SwiGLU 8/3·d

AULA 8

Famílias de modelos e atenção eficiente

PARTE 2

Apêndice matemático

Seis itens de consulta. Reconstroem, com rigor completo, todo fundamento citado no fluxo — autossuficientes para quem estuda sem ter assistido à aula.

A.1 Codificação senoidal: construção e propriedades invocado nos slides 3 e 4

NOTAÇÃO E PREMISSAS

$\text{pos} \in \{0, 1, 2, \dots\}$ índice da posição; d par, $i \in \{0, \dots, d/2-1\}$ índice do par;
 $\omega_i = 10000^{(-2i/d)}$. A base 10000 é convenção do artigo: define a faixa de períodos.

```
PE(pos)[2i]   = sin( $\omega_i \cdot \text{pos}$ )
PE(pos)[2i+1] = cos( $\omega_i \cdot \text{pos}$ )
x_pos = x_token + PE(pos)
```

1 · A FAIXA DE FREQUÊNCIAS

Com $i = 0$, $\omega_0 = 1$ e o período é $2\pi \approx 6.28$ posições. Com $i = d/2-1$, $\omega \approx 10^{-4}$ e o período é $\approx 62\ 800$. Um relógio de $d/2$ ponteiros, espaçados geometricamente.

2 · DESLOCAR POR K É LINEAR E FIXO

$$\begin{bmatrix} \sin(\omega(\text{pos}+k)) \\ \cos(\omega(\text{pos}+k)) \end{bmatrix} = \begin{bmatrix} \cos \omega k & \sin \omega k \\ -\sin \omega k & \cos \omega k \end{bmatrix} \begin{bmatrix} \sin(\omega \cdot \text{pos}) \\ \cos(\omega \cdot \text{pos}) \end{bmatrix}$$

Verificação pelas identidades de soma de arcos. A matriz depende de k e **não** de pos : existe M_k bloco-diagonal com $\text{PE}(\text{pos}+k) = M_k \cdot \text{PE}(\text{pos})$. Uma camada linear pode, em princípio, implementá-la — o argumento não diz que a rede aprenda a fazê-lo.

3 · O PRODUTO INTERNO SÓ DEPENDE DA DISTÂNCIA

$$\begin{aligned} \text{PE}(\text{pos}) \cdot \text{PE}(\text{pos}+k) &= \sum_i \left[\sin(\omega_i \text{pos}) \sin(\omega_i(\text{pos}+k)) \right. \\ &\quad \left. + \cos(\omega_i \text{pos}) \cos(\omega_i(\text{pos}+k)) \right] \\ &= \sum_i \cos(\omega_i \cdot k) \quad \# \text{ sem pos} \quad \blacksquare \end{aligned}$$

$d = 64, k = 3$
 $\text{pos} = 0 \rightarrow 25.587028547$
 $\text{pos} = 5 \rightarrow 25.587028547$
 $\text{pos} = 100 \rightarrow 25.587028547$
 $\text{pos} = 1000 \rightarrow 25.587028547$

decaimento, $d = 64$
 $k \ 0 \ 1 \ 3$
 $32.0 \ 30.9 \ 25.6$
 $k \ 10 \ 50 \ 200$
 $21.1 \ 15.7 \ 11.4$

CASOS-LIMITE

$k = 0 \rightarrow d/2$: toda codificação tem a mesma norma, $\|\text{PE}\| = \sqrt{d/2}$. Nenhuma posição é “maior”.

k grande $\rightarrow$ os cossenos dessincronizam e a soma cai, **não monotonicamente**: ela oscila em torno de um valor pequeno.

$d = 2 \rightarrow$ um par só; posições distantes por um período completo colidem.

CONTRA O CÓDIGO TABELADO (APRENDIDO)

Perde-se a etapa 2 (não existe M_k , as linhas são parâmetros independentes), a etapa 3 ($E[\text{pos}] \cdot E[\text{pos}+k]$ depende de pos) e a própria existência para $\text{pos} \geq L_{\text{max}}$. E somar não destrói o embedding porque a primeira camada é linear e pode separar subespaços — concatenar é a alternativa exata e custa dimensão: **é escolha de engenharia, não teorema**.

A.2 RoPE: a rotação e a dependência em $m - n$

invocado no slide 5

NOTAÇÃO E PREMISSAS

$q, k \in \mathbb{R}^{d_k}$ com d_k par, lidos como $d_k/2$ pares; $\theta_i = 10000^{(-2i/d_k)}$. R é aplicada **depois** de W_Q , W_K e nunca a V .

```
R_m = blocodiag( R(m*theta), ..., R(m*theta*(d_k/2-1)) )
R(a)^T R(b) = I      R(a)R(b) = R(a+b)      R(a)^T = R(-a)
```

DEMONSTRAÇÃO

$$\begin{aligned} \langle R_m q, R_n k \rangle &= (R_m q)^T (R_n k) \\ &= q^T R_m^T R_n k \\ &= q^T R(-m \cdot \theta) R(n \cdot \theta) k \\ &= q^T R((n-m) \cdot \theta) k \quad \blacksquare \end{aligned}$$

FORMA EXPLÍCITA POR PAR

$$\begin{aligned} q_i^T R(\psi) k_i &= (q_i \cdot k_i) \cos \psi - (q_i \wedge k_i) \sin \psi \\ (\tilde{q}_m, \tilde{k}_n) &= \sum_i [(q_i \cdot k_i) \cos((n-m)\theta_i) \\ &\quad - (q_i \wedge k_i) \sin((n-m)\theta_i)] \end{aligned}$$

O score de cada par é o produto interno original **girado** no plano do par: a componente alinhada entra com $\cos \psi$, a ortogonal com $\sin \psi$.

VERIFICAÇÃO NUMÉRICA (MEDIÇÃO 4)

```
d_k = 2, q = (1,0), k = (0.6,0.8), theta = 0.05
sem rotação: q.k = 0.600000
m=   3 n=   1 m-n=2 score = 0.676869
m=  10 n=   8 m-n=2 score = 0.676869
m= 500 n= 498 m-n=2 score = 0.676869
m=5000 n=4998 m-n=2 score = 0.676869
m=   3 n=   2 m-n=1 score = 0.639233
```

À mão: $\psi = -0.1, q \cdot k = 0.6, q \wedge k = 0.8 \rightarrow 0.6 \cdot 0.995004 + 0.8 \cdot 0.099833 = 0.676869 \checkmark$

POR QUE NUNCA EM V

A saída é $\sum_j A[m, j] \cdot v_j$. Rotacionar v_j por R_j soma conteúdos girados por ângulos **diferentes**, função da posição absoluta: o conteúdo passa a depender de onde o token está, e não há R comum para fatorar. Girar Q e K afeta *quem olha para quem*; girar V afetaria o *que é lido*.

CASOS-LIMITE · IMPLEMENTAÇÃO · JANELA

$m = n \rightarrow R(0) = I$: a diagonal da matriz de atenção não muda.

$\psi = \pi \rightarrow \text{score} = -q \cdot k$: pares de alta frequência carregam vizinhança, não longa distância.

d_k ímpar $\rightarrow$ não há partição em pares; a implementação exige d_k par. O $\forall d_k$ continua onde estava: a rotação preserva normas.

Sem construir a matriz: $\tilde{q}_m = q \odot \cos(m\theta) + \text{rot}(q) \odot \sin(m\theta)$. Estender a janela: θ_i/s ou reescala da base — compra-se a janela, não a competência de usá-la.

A.3 O caminho do gradiente pelo residual invocado no slide 8

NOTAÇÃO

x_l ativação na entrada do bloco l ; $J_l = \partial f_l / \partial x_l$; N blocos. As jacobianas são avaliadas no ponto de operação: o argumento é sobre a **forma** do produto.

1 · UM BLOCO · 2 · A PILHA INTEIRA

$$y = x + f(x) \quad \partial y / \partial x = I + J$$

$$\begin{aligned} \partial x_N / \partial x_0 &= \prod_l (I + J_l) \\ &= I + \sum_l J_l + \sum_{(l < l')} J_l J_{l'} + \dots + J_{(N-1)} \dots J_0 \\ &\quad \uparrow \text{caminho identidade: não multiplica nada} \end{aligned}$$

O produto deixa de ser produto: é uma **soma de caminhos**, e o mais curto é a identidade. Todos os outros termos podem ser pequenos e o gradiente ainda chega.

3 · O CONTRAFACTUAL, SEM RESIDUAL

$$\begin{aligned} \partial x_N / \partial x_0 &= \prod_l J_l \quad \|\cdot\| \leq \prod_l \|J_l\| \\ \|J\| &\approx 0.8, \quad N = 40 \rightarrow 0.8^{40} \approx 1.33 \cdot 10^{-4} \end{aligned}$$

É o produto da Aula 5 (A.2 daquele deck), agora ao longo da profundidade: o residual está para a profundidade como o gating está para o tempo.

4 · O QUE MUDA NO QUE A CAMADA APRENDE

Reescrevendo $f(x) = y - x$: o bloco aprende o **delta** sobre a representação que chegou. Logo $f = 0$ dá $y = x$, um bloco **inofensivo** — e é isso, mais do que o gradiente, que permite ir a 40 ou 100 camadas.

Corolário de inicialização: escalar a última projeção de cada sub-bloco por $1/\sqrt{2N}$ faz a pilha começar próxima da identidade.

5 · O EFEITO COLATERAL: A ESCALA CRESCE

$$\text{Var}(x_{(l+1)}) = \text{Var}(x_l) + \text{Var}(f_l(x_l))$$

A variância do fluxo residual cresce aproximadamente **linear em l**. Duas consequências: (i) alguma normalização é obrigatória dentro do bloco (A.4); (ii) em pré-norm a última soma não é normalizada por ninguém, o que torna a norma final obrigatória (A.5).

CASOS-LIMITE

$J_l = 0$ para todo $l \rightarrow \partial x_N / \partial x_0 = I$: o gradiente atravessa intacto e o treino continua possível. Sem residual, seria zero.

$\|J_l\| \gg 1 \rightarrow$ o residual **não** protege: é remédio para desaparecimento, não para explosão. O outro lado é tratado por normalização e recorte de gradiente.

$N = 1 \rightarrow I + J$, e a discussão de profundidade não se aplica.

A.4 LayerNorm e RMSNorm invocado no slide 9

ESTATÍSTICAS E DEFINIÇÕES

$$\begin{aligned}\mu &= (1/d) \cdot \sum_i x_i && \# \text{ sobre as DIMENSÕES} \\ \sigma^2 &= (1/d) \cdot \sum_i (x_i - \mu)^2 \\ \text{RMS}(x) &= \sqrt{(1/d) \cdot \sum_i x_i^2}\end{aligned}$$

$$\begin{aligned}\text{LayerNorm}(x) &= \gamma \odot (x - \mu \cdot 1) / \sqrt{(\sigma^2 + \epsilon)} + \beta \\ \text{RMSNorm}(x) &= \gamma \odot x / \text{RMS}(x)\end{aligned}$$

1 · INVARIÂNCIA A ESCALA (AS DUAS TÊM)

$\text{RMS}(a \cdot x) = a \cdot \text{RMS}(x) \Rightarrow (ax) / \text{RMS}(ax) = x / \text{RMS}(x)$. É esta propriedade que corta o crescimento de escala de A.3: qualquer que seja a escala acumulada, a entrada do ramo chega normalizada — e os escores $QK^T / \sqrt{d_k}$ não herdam a escala da profundidade.

2 · INVARIÂNCIA A DESLOCAMENTO (SÓ LN)

$$\begin{aligned}\mu(x + c \cdot 1) &= \mu + c \quad \rightarrow \quad (x + c \cdot 1) - (\mu + c) \cdot 1 = x - \mu \cdot 1 \\ \sigma^2(x + c \cdot 1) &= \sigma^2 \quad \Rightarrow \quad \text{LayerNorm}(x + c \cdot 1) = \text{LayerNorm}(x) \quad \blacksquare \\ \text{RMS}(x + c \cdot 1) &\neq \text{RMS}(x) \quad \text{em geral}\end{aligned}$$

É isto que se perde ao remover a média — e, empiricamente, perde-se pouco: a rede aprende a centrar onde precisar.

O QUE A SEGUNDA ECONOMIZA

parâmetros	$2d \rightarrow d$ (só γ). Metade.
operações	duas reduções + d subtrações $\rightarrow$ uma redução; $\sim 5d$ contra $\sim 3d$.
memória	uma estatística a menos por vetor no backward.
não economiza	a redução continua sendo sincronização sobre d .

O GRADIENTE É UMA PROJEÇÃO

$$\partial \gamma_i / \partial x_j = (1/\text{RMS}) \cdot (\delta_{ij} - x_i x_j / (d \cdot \text{RMS}^2))$$

O segundo termo subtrai a projeção radial: o gradiente que passa pela norma é **ortogonal a x** . A norma remove um grau de liberdade — a escala — do problema de otimização. LN faz o mesmo e remove também a componente constante.

ISTO NÃO É BATCHNORM · CASOS-LIMITE

LN/RMSNorm reduzem sobre **a linha**; BatchNorm sobre a coluna. Nenhum exemplo influencia outro, não há estatística móvel, e lote 1 é idêntico ao treino — necessário para geração autorregressiva.

$x = 0 \rightarrow \epsilon$ evita divisão por zero; sem ele, NaN. Valores típicos $10^{-5} \dots 10^{-6}$.

$d = 1 \rightarrow$ LN é destrutiva (saída constante β); RMSNorm devolve $\pm \gamma$, preservando o sinal.

$x = c \cdot 1 \rightarrow$ LN devolve β (a informação some); RMSNorm devolve $\gamma \cdot \text{sign}(c)$.

A.5 Pré-norm × pós-norm: a diferença algébrica invocado no slide 10

AS DUAS CONVENÇÕES

pós-norm: $x_{(l+1)} = \text{LN}(x_l + f_l(x_l))$

pré-norm: $x_{(l+1)} = x_l + f_l(\text{LN}(x_l))$

As peças são as mesmas. Muda o que fica **dentro** e o que fica **fora** da soma. LN denota a norma escolhida — o argumento não depende de qual.

DERIVAÇÕES

pós: $\partial x_{(l+1)} / \partial x_l = J_{\text{LN}} \cdot (I + J_f)$
 $\partial x_N / \partial x_0 = \Pi_l [J_{\text{LN}}^l \cdot (I + J_f^l)]$

pré: $\partial x_{(l+1)} / \partial x_l = I + J_f \cdot J_{\text{LN}}$
 $\partial x_N / \partial x_0 = I + \sum_l (J_f^l \cdot J_{\text{LN}}^l) + \dots$

Em pós-norm o termo identidade continua lá, mas **multiplicado por um J_{LN} a cada camada**: a pilha herda um produto de fatores de escala e volta, em grau menor, ao regime de A.3. Em pré-norm o caminho identidade **sobrevive intacto**, e a escala do gradiente é aproximadamente independente da profundidade.

O CASO-LIMITE QUE SEPARA AS DUAS

bloco com $f = 0$:

pós: $x_{(l+1)} = \text{LN}(x_l) \neq x_l$ # NÃO é inofensivo

pré: $x_{(l+1)} = x_l$ # É inofensivo

Em pós-norm, mesmo um bloco que não faz nada **altera** a ativação, porque a norma está no caminho principal. A diferença é exata, não heurística.

POR QUE A NORMA FINAL É OBRIGATÓRIA

Em pré-norm a última operação é uma **soma**: ninguém normaliza x_N . Com a variância crescendo com a profundidade (A.3, etapa 5), os logits herdam a escala acumulada de N blocos: o softmax satura, o gradiente da cabeça encolhe, e o treino diverge **sem levantar exceção**.

logits = Linear(LN(x_N)) # antes da cabeça

Em pós-norm ela não é necessária — a saída do último bloco já passou por uma norma. É o detalhe que falta em quase todo diagrama publicado, e é um TODO do Lab 2.

PREÇO E CASOS-LIMITE

Em pilhas rasas, pós-norm rende um pouco mais: normalizar a saída funciona como regularização. A inversão acompanhou a profundidade — não é certo contra errado.

$N = 1 \rightarrow$ as duas ordens diferem pouco; a escolha é irrelevante.

$J_{\text{LN}} \approx I \rightarrow$ as convenções convergem; é o regime que o aquecimento tenta produzir artificialmente.

A.6 SwiGLU e a contagem de parâmetros do bloco invocado nos slides 11 e 12

NÃO LINEARIDADES E VARIANTES

```

GELU(x) = x · Φ(x)           SiLU(x) = x · σ(x)
SiLU'(x) = σ(x) · ( 1 + x · (1 - σ(x)) )

FFN_2017(x) = W_2 · GELU(W_1 x)      d_ff = 4 · d
SwiGLU(x)   = (SiLU(x W_gate) ⊙ (x W_up)) W_down
  
```

A SiLU é suave em toda a reta, não monótona, com mínimo em $x \approx -1.2785$ (valor ≈ -0.2784); derivada não nula à esquerda, sem neurônios permanentemente mortos. O gating é um **reostato por dimensão** — um ramo decide o quê, o outro quanto passa; zera um canal sem que nenhum peso seja zero.

DERIVAÇÃO DA PROPORÇÃO 8/3

```

FFN:      2 · d · d_ff = 2 · d · (4d) = 8 · d²
SwiGLU:   3 · d · d_ff
paridade: 3 · d · d_ff = 8 · d² ⇒ d_ff = (8/3) · d ■
  
```

Não é estética nem cabala: é o número que paga a terceira matriz sem inflar a contagem.

O BLOCO INTEIRO · FLOPS POR TOKEN

```

atenção (W_Q, W_K, W_V, W_O) → 4 · d²    4 194 304
FFN      → 8 · d²    8 388 608
bloco (d = 1024) → 12 · d²  12 582 912
  
```

A contagem é **quadrática**: dobrar d para 2048 dá $\approx 50,3$ M por bloco — quatro vezes, não duas. Nos FLOPs, some o termo $O(n \cdot d)$ da própria matriz de atenção: contexto curto, o FFN domina; contexto longo, a atenção passa a dominar (volta na Aula 12).

ENCODER × DECODER: 4 CONTRA 8

```

enc:  4 · d · d_k · h + 2 · d · d_ff + 5d + d_ff    1 atenção
dec:  8 · d · d_k · h + 2 · d · d_ff + 7d + d_ff    2 atenções
  
```

Um bloco de decoder *encoder-decoder* custa o dobro em atenção (a cruzada), e o $5d \rightarrow 7d$ conta a terceira LayerNorm. **Antes de comparar duas contagens, verifique se as duas contam vieses:** com vieses o decoder-only dá $12d^2 + 9d$; sem, $12d^2 + 4d$.

CASOS-LIMITE E USO NO LAB 2

$W_{\text{gate}} = 0 \rightarrow$ bloco morto, que o residual torna inofensivo em vez de destrutivo.

$d_{\text{ff}} = d \rightarrow$ sem expansão, a não linearidade opera num espaço estreito; a expansão é o que dá capacidade por posição.

Prever a contagem antes de imprimir `sum(p.numel() ...)`: divergência > 5% denuncia peça a mais ou a menos — a mais comum é a cabeça de saída não compartilhar pesos com o embedding.